

BİL214 - System Programming

TOBB ETU

2024-25 Spring

Çiğdem ADA

Course Outline

- Introduction to Linux
 - Basic Linux Shell Commands, Simple Bash Scripting
- Advanced C
 - Conditionals, Loops, Array, Pointer, Struct, Dynamic Memory Allocation
- Introduction to Operating System Concepts
 - Processes and Inter-process Communication

References

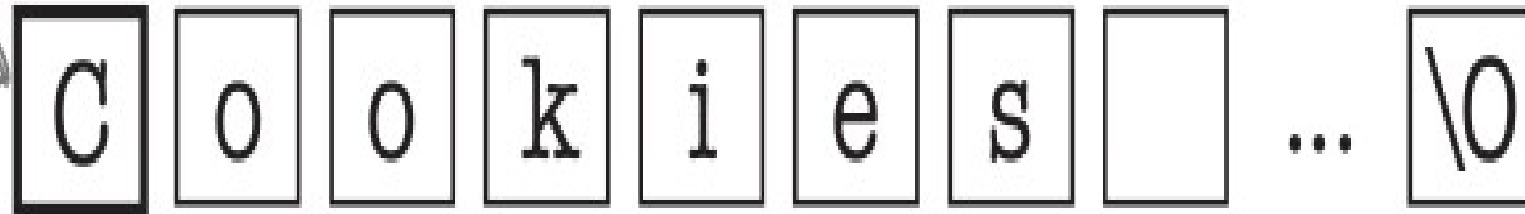
- <https://linuxhint.com/>
- <https://www.geeksforgeeks.org/>
- BİL 214 Lecture Notes of Previous Semesters
- Head First C by David Griffiths and Dawn Griffiths – O'Reilly ISBN: 978-1-449-39991-7
- C How To Program by Paul Deitel and Harvey Deitel – Pearson ISBN-13: 978-0-13-299044-8

Strings

- Strings are like character arrays...

The quote variable will represent the address of the first character in the string.

```
char quote[] = "Cookies make you fat";
```



Null character '\0'

But what happens when C wants to read the contents of the string? Say it wants to print it out. Now, in a lot of languages, the computer keeps pretty close track of the size of an array, but C is more low-level than most languages and can't always work out exactly *how long* an array is. If C is going to display a string on the screen, it needs to know when it gets to the end of the character array. And it does this by adding a **sentinel character**.

sizeof() operator for strings

You can either call it with a data type or with a piece of data.

On most machines, this $\rightarrow$ `sizeof(int)`
will return the value 4.
`sizeof("Turtles!")` $\leftarrow$ This will return 9, which is 8
characters plus the `\0` end character.

strlen() - Finding String Length

- The strlen() function returns the length of a string (excluding the null character).

```
#include <stdio.h>
#include <string.h>

int main() {
    char str[] = "Hello";
    printf("String length: %d", (int)strlen(str));
    return 0;
}
```

- Output: String length: 5

```
int my_strlen(char s[])
{
    int i = 0;
    while (s[i] != '\0')
        i++;
    return i;
}
```

How does %s work?

- The computer will set aside space on the stack for each of the characters in the string, plus the '\0' end character.
- Every time the quote variable is used in the code, the computer will substitute it with the address of the first character in the string.
- `printf("%s", quote)` → starts from the quote and prints everything until it finds null character.
- See `string_cookie.c`

String Literals

- Alternatively, in order to define string literals (constant strings) you can use `char*`. If you use `char*` be careful and don't try to change it or use `const char*`.
- Similar to Java's immutable `String` class...
- See `jqk.c`

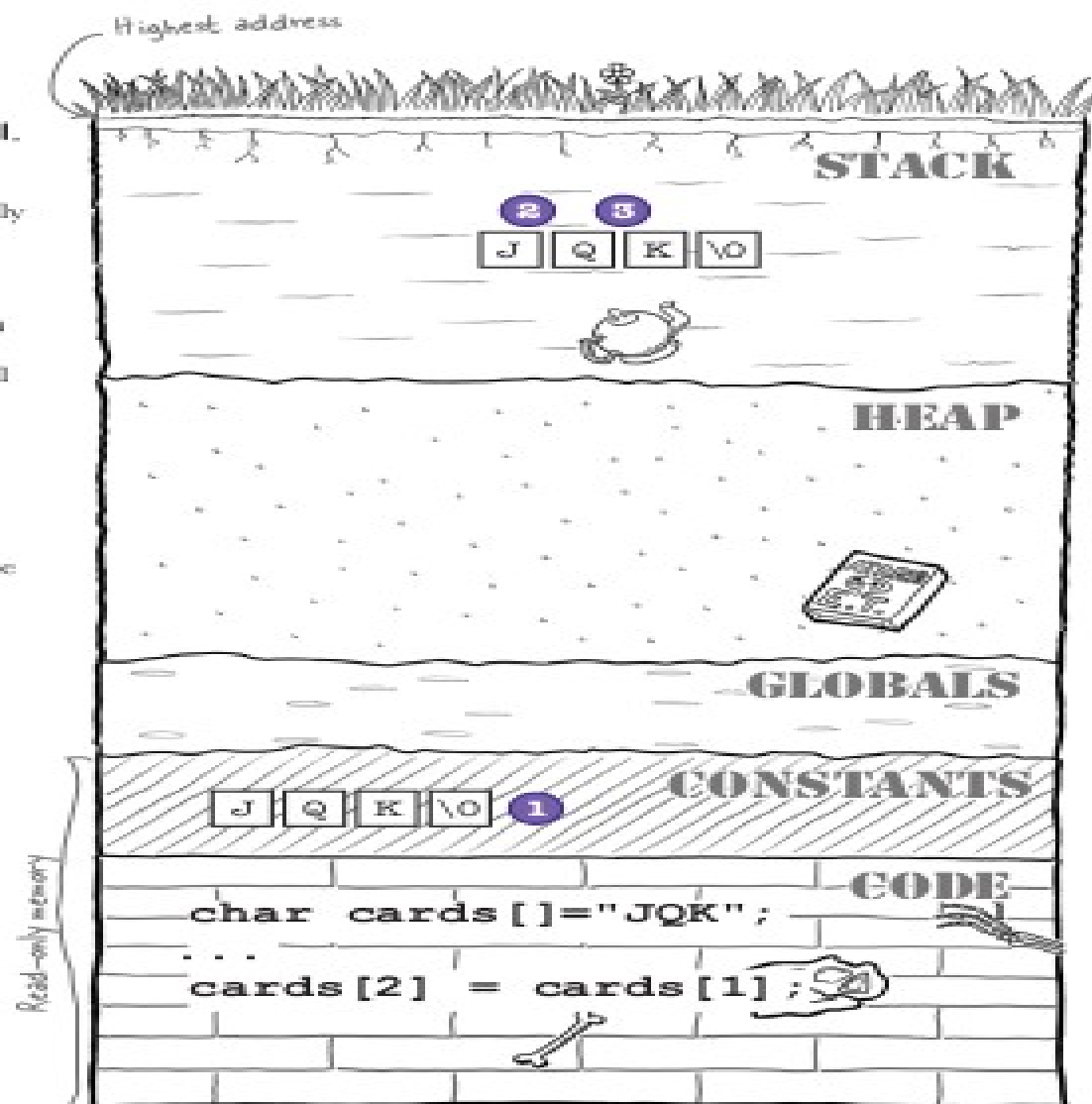


In memory: `char cards[]="JQK";`

We've already seen what happens with the *broken code*, but what about our new code? Let's take a look.

- 1 The computer loads the string literal.**
As before, when the computer loads the program into memory, it stores the constant values—like the string “JQK”—into read-only memory.
- 2 The program creates a new array on the stack.**
We're declaring an array, so the program will create one large enough to store the “JQK” string—four characters' worth.
- 3 The program initializes the array.**
But as well as allocating the space, the program will also **copy the contents** of the string literal “JQK” into the stack memory.

So the difference is that the original code used a pointer to point to a read-only string literal. But if you initialize an array with a string literal, you then have a *copy* of the letters, and you can change them as much as you like.





In memory: `char *cards="JQK";`

To understand why this line of code causes a memory error, we need to dig into the memory of the computer and see exactly what the computer will do.

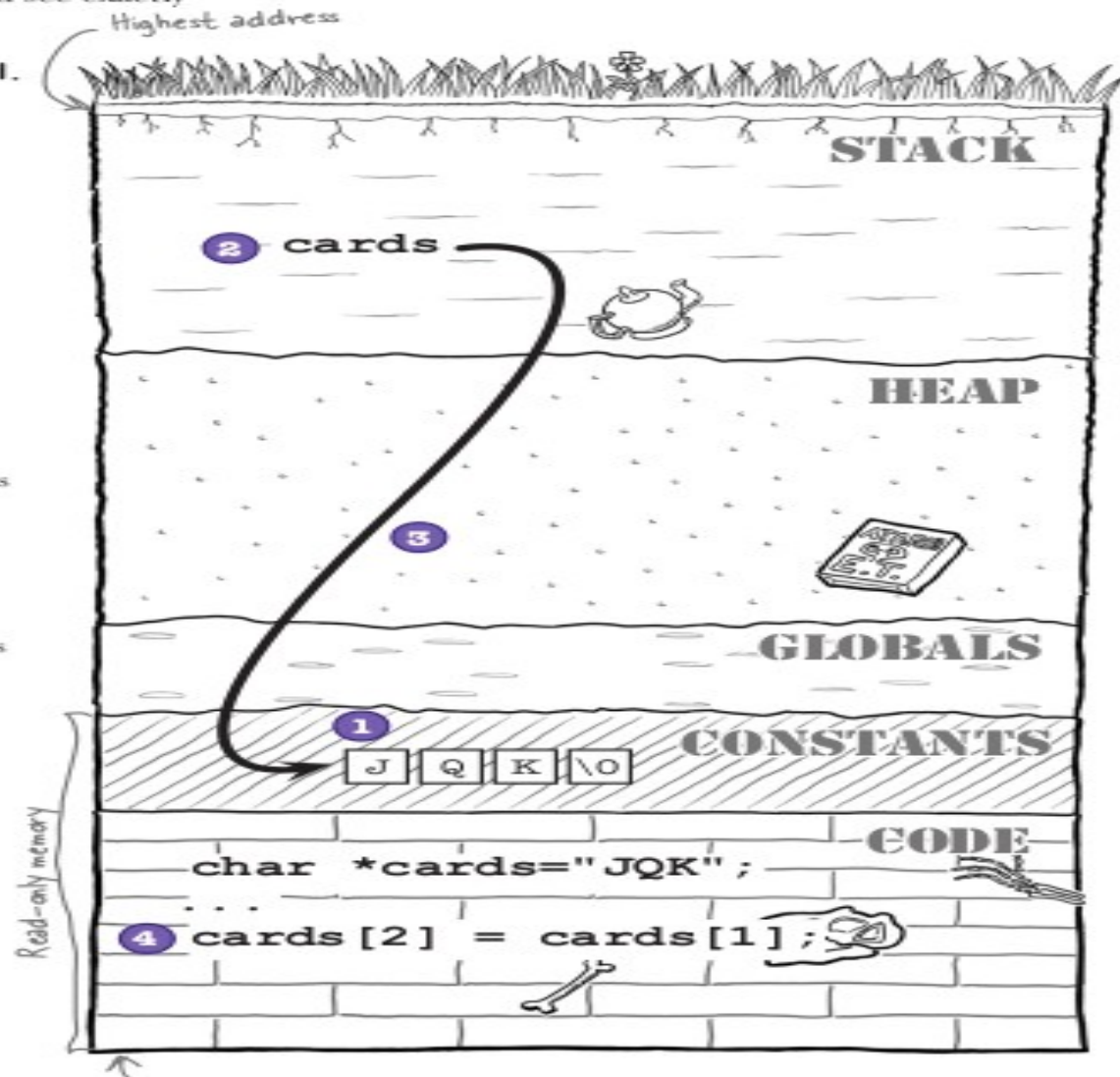
- 1 The computer loads the string literal.**
When the computer loads the program into memory, it puts all of the constant values—like the string literal “JQK”—into the constant memory block. This section of memory is **read only**.
- 2 The program creates the cards variable on the stack.**
The stack is the section of memory that the computer uses for local variables: variables inside functions. The cards variable will live here.
- 3 The cards variable is set to the address of “JQK.”**
The cards variable will contain the address of the string literal “JQK.” String literals are usually stored in read-only memory to prevent anyone from changing them.
- 4 The computer tries to change the string.**
When the program tries to change the contents of the string pointed to by the cards variable, it can't; the string is read-only.

I can't update that, buddy. It's in the constant memory block, so it's read-only.



So the problem is that string literals like “JQK” are held in read only memory. They're constants.

But if that's the problem, how do you fix it?



One way to avoid this problem in the future is to never write code that sets a simple `char` pointer to a string literal value like:

```
char *s = "Some string";
```

There's nothing wrong with setting a pointer to a string literal—the problems only happen when you try to *modify* a string literal. Instead, if you want to set a pointer to a literal, always make sure you use the `const` keyword:

```
const char *s = "some string";
```

That way, if the compiler sees some code that tries to modify the string, it will give you a compile error:

```
s[0] = 'S';
```

```
monte.c:7: error: assignment of read-only location
```

How do you pass a string to a function?

```
#include <stdio.h>

void fortune_cookie(char msg[]);

int main()
{
    char quote[] = "Cookies make you fat";
    fortune_cookie(quote);
    return 0;
}

void fortune_cookie(char msg[])
{
    printf("Message reads: %s\n", msg);
}
```

Just passing a pointer

- See `pass_cookie.c`
- Even though it looked like you were passing a string to the `fortune_cookie()` function, you were actually just passing a pointer to it.
- And that's why the `sizeof` operator returned a weird result. It was just returning the size of a pointer to a string.
- See `pass_cookie_ptr.c`